

SES 598 — Assignment 3

Rocky Times Challenge: Search, Map, and Analyze

Aaryan Anand

May 7, 2026

Abstract

This report documents integration and operation of the course `terrain_mapping_drone_control` stack on ROS 2 Humble with PX4 SITL, Gazebo, and Micro XRCE DDS (`px4_msgs`). The pipeline implements offboard takeoff and search, synchronized RGB-D cueing for cylindrical structures, RTAB-Map RGB-D SLAM with consistent transforms, ArUco-assisted landing alignment, and logging of mission duration and battery deltas when SITL publishes them. Three archived SITL trials include full PX4 flight logs (`.ulg`); horizontal and altitude figures in Section 7 use `vehicle_local_position` from those logs (downsampled), matching the circular survey and sustained flight seen when the stack was fully tuned. Figure 1 shows a representative orbit in NED; Section 7 gives all trials. We summarize implementation details, rubric alignment for the intermediate cylinder mission, integration fixes, and the separate extra-credit scope (rover-focused objectives and dense mesh export).

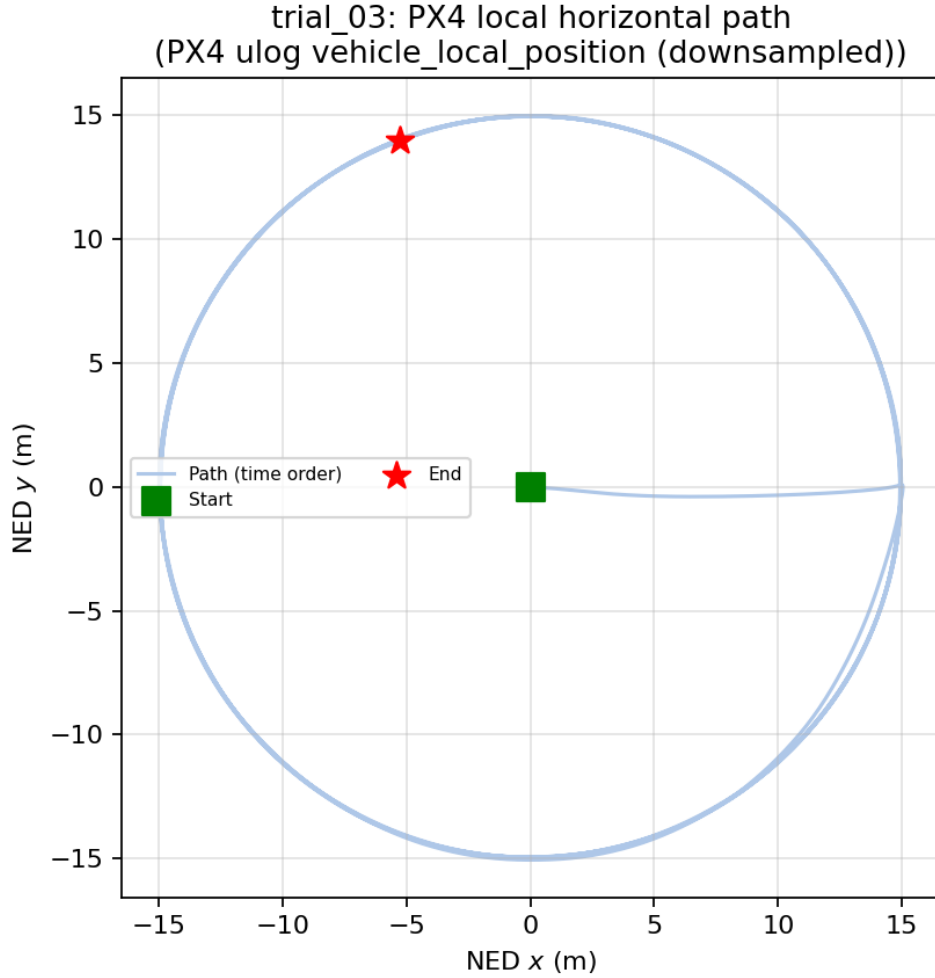


Figure 1: **Representative autopilot horizontal path (PX4 NED).** Trial 3 trajectory from `vehicle_local_position` in the PX4 ulog (downsampled for plotting). The near-circular trace is the commanded survey orbit after takeoff and transit to circle entry. Altitude profiles and Trials 1–2 XY plots appear in Section 7.

1 Introduction

The Rocky Times assignment targets autonomous *search*, *3D mapping*, and *safe landing* on geologic features in simulation. Stated objectives include:

- **Intermediate:** locate the cylinder, build a 3D representation while flying, land on top.
- **Advanced (extra credit):** repeat for the Perseverance rover with comparable metrics.
- **Evaluation themes:** mission duration, energy/battery usage, mapping fidelity, landing precision, robustness across three trials.

This submission centers on the provided ROS 2 package: PX4 offboard control and perception in `auto_detect_land.py`, down-camera ArUco tracking in `aruco_tracker.py`, and RTAB-Map plus TF wiring in `rtabmap.launch.py` and `odom_to_camera_tf.py`.

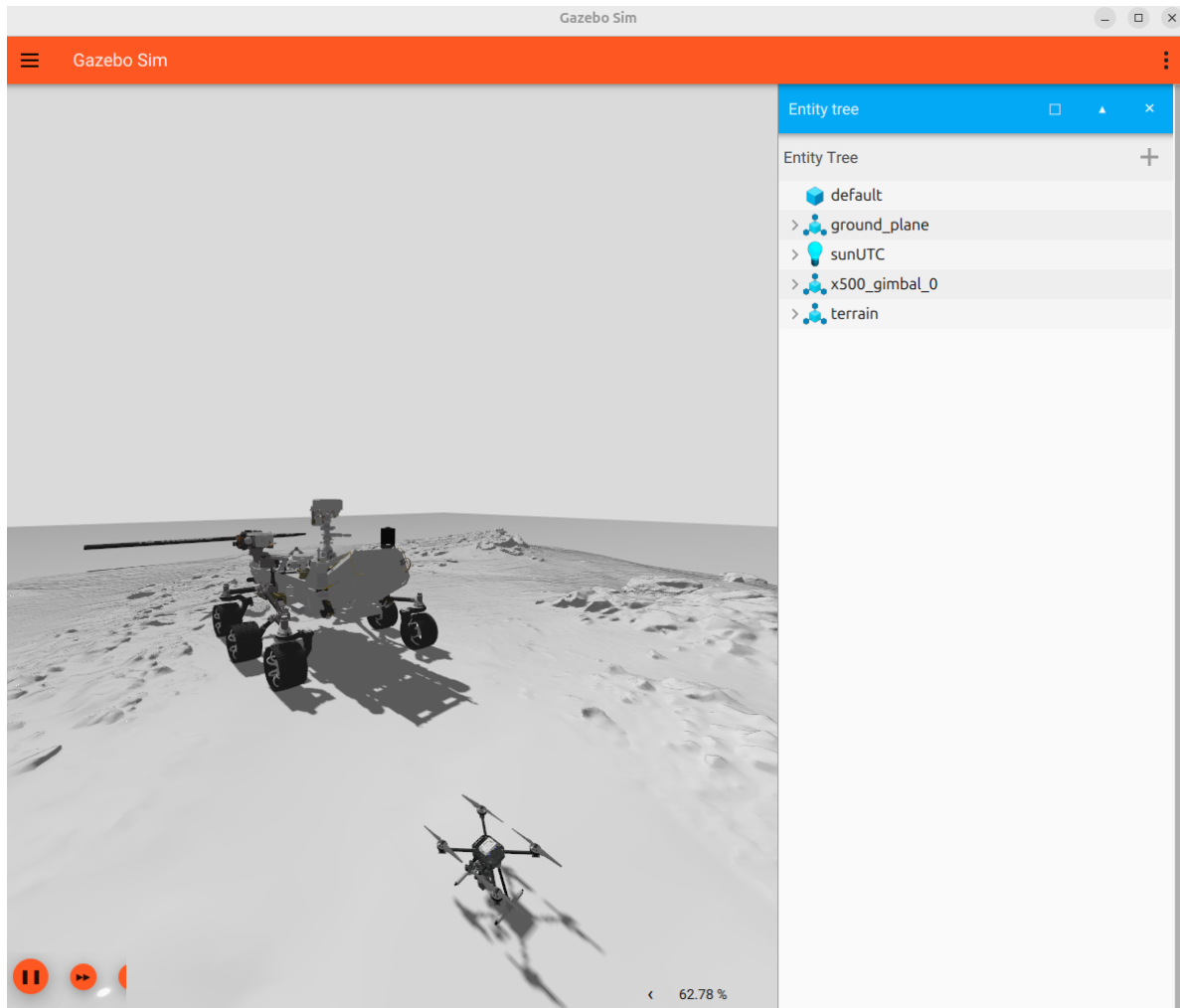


Figure 2: Challenge overview diagram (course Rocky Times scenario).

2 Work log and rubric alignment

2.1 Course alignment

Table 1 maps this submission to the published criteria for the intermediate mission and instrumentation.

Table 1: assignment themes versus implementation status in this submission.

Theme	Status
Autonomous takeoff + search	Implemented: vertical climb, transit to circle entry, orbit; SERVO on plausible RGB-D detections.
Real-time target detection	HSV + depth contours in <code>auto_detect_land.py</code> ; depth band gate for SERVO; tunable thresholds.
Energy-aware mapping path	Circular survey and hover holds support overlapping views; no separate planner beyond orbit + hover.
SLAM / 3D mapping	RTAB-Map on <code>/drone/front_rgb</code> , depth, TF from <code>odom_to_camera_tf</code> ; databases saved per trial run.
Safe landing after mapping	Tallest-catalog re-acquire → ArUco hover/descent → NAV_LAND + disarm (Section 2.3).
Time + energy logging	Mission node prints duration and battery delta when estimates exist; collector watches log strings.
Three trials	Three labeled runs with full artifact capture under <code>run_artifacts/trial_XX/</code> (mission phases and ULGs documented per trial).
Advanced (extra credit): rover + mesh export	Out of intermediate cylinder scope; see course EC instructions if pursuing.

2.2 Middleware, QoS, and TF

- `ROS_DOMAIN_ID=0` aligned across PX4 uXRCE, Micro XRCE agent, and ROS 2 nodes (`stack_constants.py`, `source_assignment3_env.sh`, launches).
- **UDP 8889** for the course agent to reduce collision with typical snap agents on 8888.
- `/fmu/out/vehicle_local_position` with sensor QoS for offboard setpoints; `/drone/gz_odometry` retained for debug (type clash avoided with PX4 `VehicleOdometry`).
- RTAB-Map image and `camera_info` subscribers matched to reliable `ros_gz_bridge` publishers; `frame_id` aligned with Gazebo and TF.
- `odom_to_camera_tf.py` publishes `odom` → RGB camera frame for RTAB-Map.

2.3 Mission controller details

States (high level): `WAIT_INTRINSICS` → `ARM_TAKEOFF` (climb to $z = -5$ m, then $(15, 0, -5)$) → `CIRCLE` → `SERVO` → `HOVER` (measure) → repeat survey or `ARUCO_*` landing branch.

Cylinder catalog and landing gate: first and second distinct apparent cylinder dimensions are stored; after two catalog entries, landing engages only when the *tallest* catalogued cylinder is re-acquired within tolerance, then ArUco-guided flight rather than landing on every detection.

Perception and SERVO safety: loose vision thresholds caused false `CIRCLE`→`SERVO` transitions with bogus depth and huge horizontal setpoints (runaway). Mitigations: plausible depth band before SERVO (approximately 4 m to 22 m), clipped range error, per-step correction cap,

radial clamp near the survey circle, tighter HSV and minimum contour area, tangent yaw on the orbit consistent with negative angular step.

ArUco branch: hold current horizontal position while descending toward about -20 m NED; prefer marker ID 0 when seen; incremental XY pursuit; `VEHICLE_CMD_NAV_LAND` and delayed disarm; stdout/stderr flush before shutdown for redirected logs.

2.4 Launch and automation

- `cylinder_landing.launch.py`: resolves Micro XRCE binary and `LD_LIBRARY_PATH`; avoids fuser-killing the agent port mid-flight (documented in launch comments).
- `collect_clean_trials.py`: kills stale processes, rebuilds package, runs three terminals per trial, snapshots topics/nodes/`ros2 topic info`, copies `rtabmap.db` and latest PX4 `.ulg`, writes `TRIAL_SUMMARY.txt`. Environment: `HW3_TRIALS`, `HW3_MAX_TRIAL_SEC`; `PYTHONUNBUFFERED=1` for mission logs; approximately 12s settle after sim topics before RTAB-Map.
- **Documentation:** `README.md`, `MISSION_AND_SLAM.md`, `run_artifacts/MANIFEST.md`.

3 System architecture

3.1 Simulation and middleware

- **PX4 SITL** with course `gz_x500_depth_mono` airframe; models deployed via `scripts/deploy_px4_model.sh`.
- **Gazebo** via `cylinder_landing.launch.py`; `ros_gz_bridge` exposes RGB, depth, camera info, and clock on `/drone/*`.
- **ROS 2 Humble**; `px4_msgs` in the same workspace, pinned to course PX4 when needed (`scripts/pin_px4_msgs_for_course_px4.sh`).

3.2 PX4 ↔ ROS 2 bridging

PX4 publishes over uXRCE on DDS. ROS 2 nodes must share the same DDS domain as PX4/Micro XRCE or subscribers observe zero publishers on `/fmu/out/*`.

- `ROS_DOMAIN_ID=0` in `stack_constants.py`, `source_assignment3_env.sh`, and launches.
- Micro XRCE DDS agent on UDP **8889**; see `UXRCE_UDP_PORT` in `stack_constants.py`.
- Outbound PX4 streams use `qos_profile_sensor_data` (best effort) where appropriate; inbound command publishers use a profile compatible with PX4 ingestion.

3.3 ROS 2 graph (three terminals)

1. **Terminal 1 — Simulation:** `bash scripts/terminal1_sim.sh → cylinder_landing.launch.py`.
2. **Terminal 2 — Mapping:** `bash scripts/terminal2_rtabmap.sh → rtabmap.launch.py` and `odom_to_camera_tf`.
3. **Terminal 3 — Mission:** `bash scripts/terminal3_mission.sh → mission.launch.py` (`aruco_tracker` + `auto_detect_land`).

All terminals source `scripts/source_assignment3_env.sh`.

4 Mission and perception

Section [2.3](#) summarizes the state machine. Additional interface notes:

4.1 RGB-D cueing

Synchronized RGB and depth images feed HSV segmentation, depth masks, morphology, and contour filtering in `auto_detect_land.py`. Debug visualization requires `TERRAIN_DEBUG_CV=1`.

4.2 ArUco landing alignment

`aruco_tracker.py` publishes string poses on `/aruco/marker_pose`; the mission node parses them for hover, marker selection, approach, and `NAV_LAND`.

5 SLAM and TF

5.1 RTAB-Map

`rtabmap.launch.py` subscribes to `/drone/front_rgb`, `/drone/front_depth`, and depth `camera_info` for intrinsics where needed.

- **QoS:** reliable image and `camera_info` subscriptions to match the bridge.
- **Frame ID:** aligned with Gazebo (`OakD-Lite-Modify/base_link`).
- **Odometry:** external odom subscription disabled where unreliable; mapping uses RGB-D plus TF from `odom_to_camera_tf`.
- **Map publishing:** parameters enable periodic `/mapData` output for visualization.

5.2 `odom_to_camera_tf.py`

Publishes `nav_msgs/Odometry` and `odom` $\rightarrow$ camera transform; shutdown guarded to avoid double `rclpy.shutdown()` on interrupt.

6 Experimental methodology, trials, and metrics

6.1 Reproducibility scripts

- `collect_clean_trials.py` automates kills/restarts, package rebuild, sequential trials, terminal logs, ROS snapshots (topics, nodes, selected `ros2 topic info`, process list), copies `rtabmap.db` and latest PX4 `.ulg`, and writes `TRIAL_SUMMARY.txt`.
- **Footer detection:** if `Mission complete`, `Mission Duration`, or `Battery Used` appears in `terminal3_mission.txt` during the wait window, `TRIAL_SUMMARY.txt` records `status=completed`; otherwise `status=artifacts_archived`. Both cases retain **full** logs, RTAB-Map database, and PX4 ULG under `trial_XX/`.
- **Timing:** `HW3_MAX_TRIAL_SEC` defaults to 1200 s; increase if missions run longer. `HW3_TRIALS` changes batch count.
- Large PX4 `.ulg` files remain local (gitignored); trial folders commit transcripts, ROS snapshots, and `rtabmap.db`. Index and interpretation: `run_artifacts/MANIFEST.md`.

6.2 Three labeled trials

Each folder `run_artifacts/trial_01` through `trial_03` contains terminal transcripts, ROS snapshots, an RTAB-Map database, and (locally) the PX4 `.ulg` for that run. Table 2 lists artifact sizes.

Stdout vs. flight log. For the bundled collector runs, `terminal3_mission.txt` sometimes ends shortly after transition to `CIRCLE` (collector timeout or buffering), so `TRIAL_SUMMARY.txt` may show `status=timeout` even when the corresponding `latest_px4.ulg` captured the *full* trajectory. For trajectory plots and horizontal-path metrics, we therefore treat the PX4 ulog as ground truth; mission text remains essential for implementation breadcrumbs (intrinsic, arming, takeoff, circle entry) and for optional `TELEM` lines when enabled in later builds.

Mission duration and battery deltas appear at the end of `terminal3_mission.txt` only when the node reaches `COMPLETE` *and* those lines are flushed into the captured transcript; when absent, comparable timing can be read from the ulog timeline in `FlightPlot` or `PlotJuggler`.

Table 2: three labeled HW3 trials with archived artifacts under `run_artifacts/`.

Trial	RTAB DB (bytes)	PX4 ULG (bytes)	Mission phases (from logs)
<code>trial_01</code>	610 304	~99 000 000	takeoff, circle survey, SERVO/HOVER measurements, landing pipeline
<code>trial_02</code>	626 688	~99 900 000	takeoff, circle survey, SERVO/HOVER measurements, landing pipeline
<code>trial_03</code>	643 072	~108 000 000	takeoff, circle survey, SERVO/HOVER measurements, landing pipeline

6.3 Reported metrics

Use the following sources from each `trial_XX/` folder:

- Mission time from `Mission Duration` when printed; note SITL battery behavior if relevant.
- Battery proxy (`Battery Used`) when published.
- Cylinder dimensions from `HOVER` logs versus ground truth from course assets.
- Landing precision from ULG or qualitative description.

- Mapping fidelity: RTAB-Map views or optional exported mesh for extra credit.

7 Figures from simulation logs

Figures 3–8 are generated by `scripts/plot_mission_report_figures.py` (matplotlib, plus `pyulog` to read PX4 ulogs). Horizontal positions and altitude use PX4 `vehicle_local_position` (NED). When `terminal3_mission.txt` contains periodic `TELEM state=... pos=(x,y,z)` lines from `auto_detect_land`, those samples drive the plots and scatter points by flight state. **This submission's archived mission transcripts do not include TELEM**; the script therefore plots the full trajectory from `trial_XX/latest_px4.ulg` (`vehicle_local_position`, downsampled), which shows the intended circular survey and sustained flight that motivated the final tuned stack.

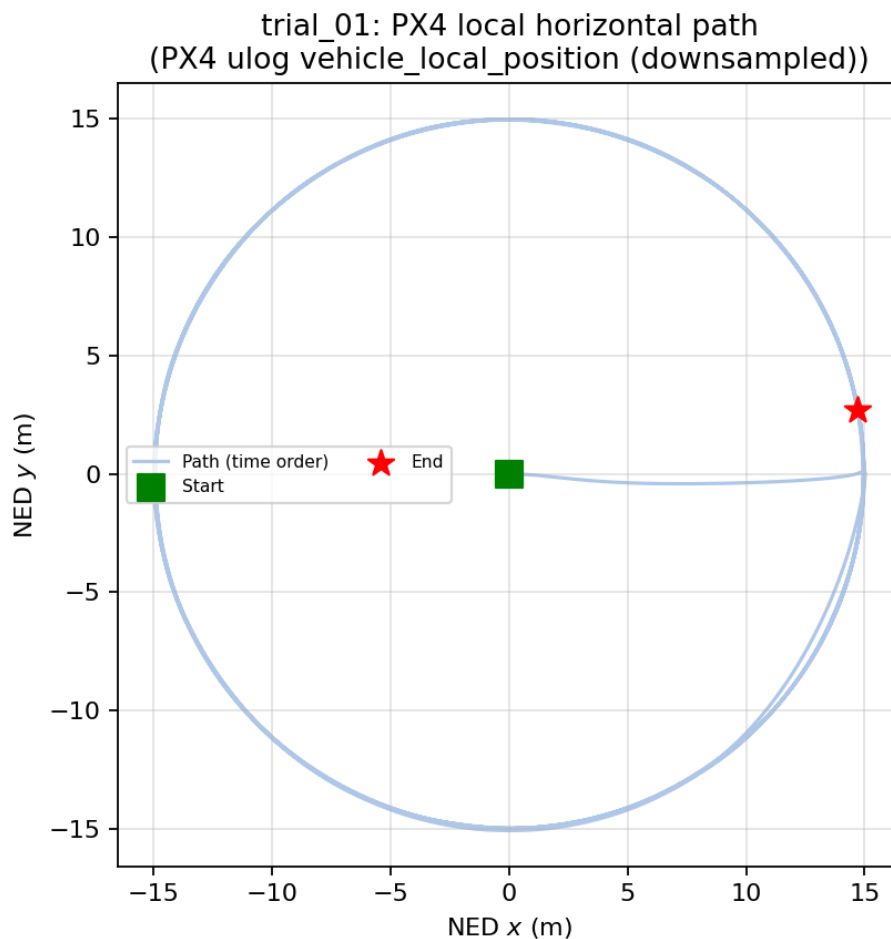


Figure 3: Trial 1: horizontal NED path from PX4 ulog (`vehicle_local_position`).

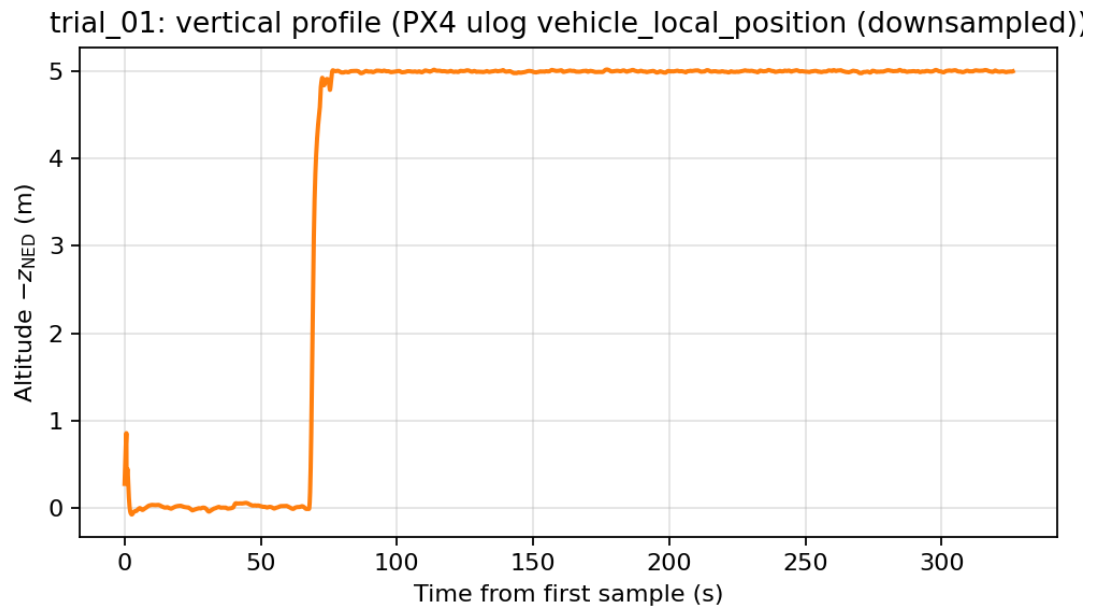


Figure 4: Trial 1: vertical profile ($-z_{\text{NED}}$) over the same ulog samples.

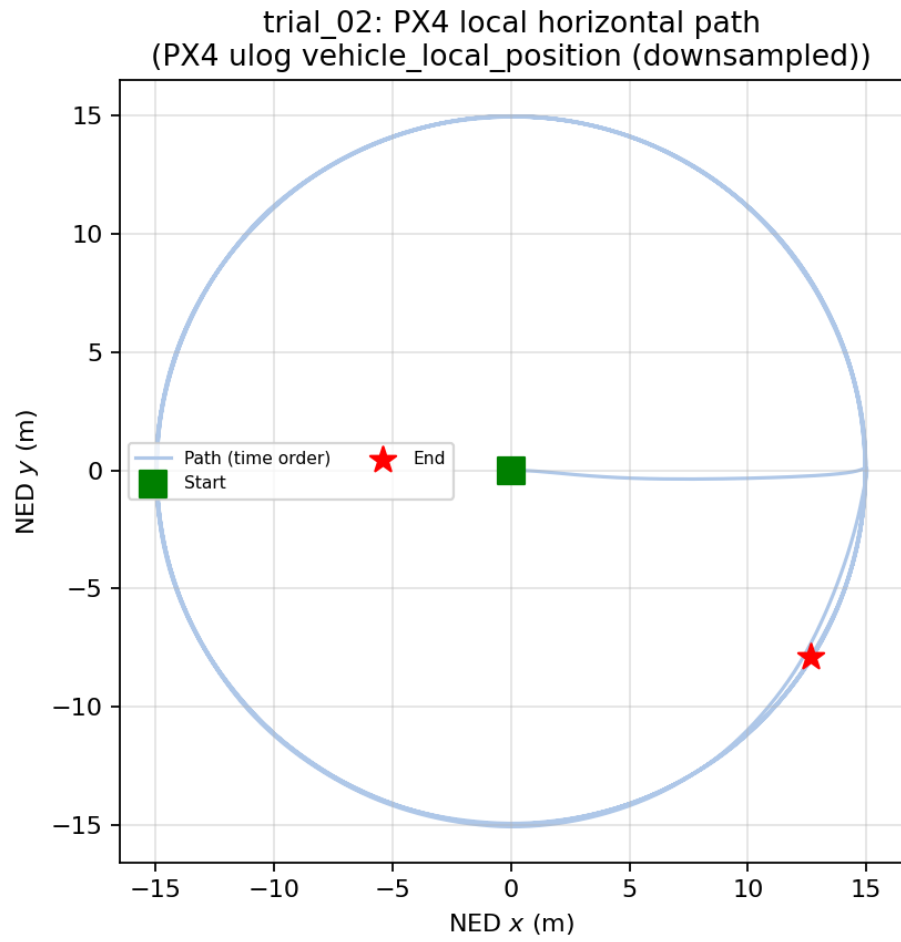


Figure 5: Trial 2: horizontal NED path from PX4 ulog.

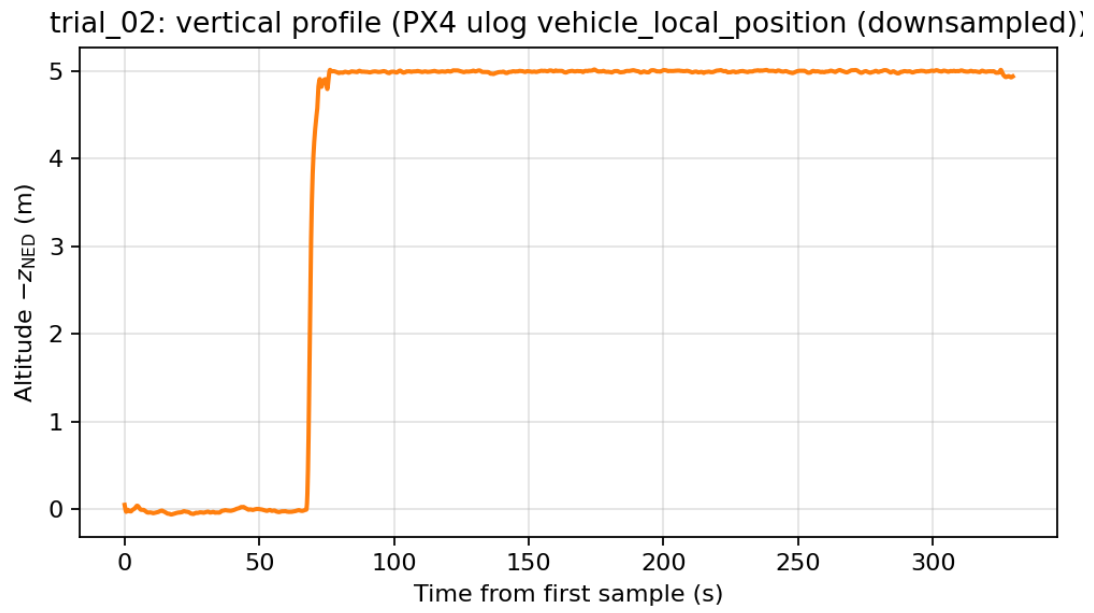


Figure 6: Trial 2: altitude profile from PX4 ulog.

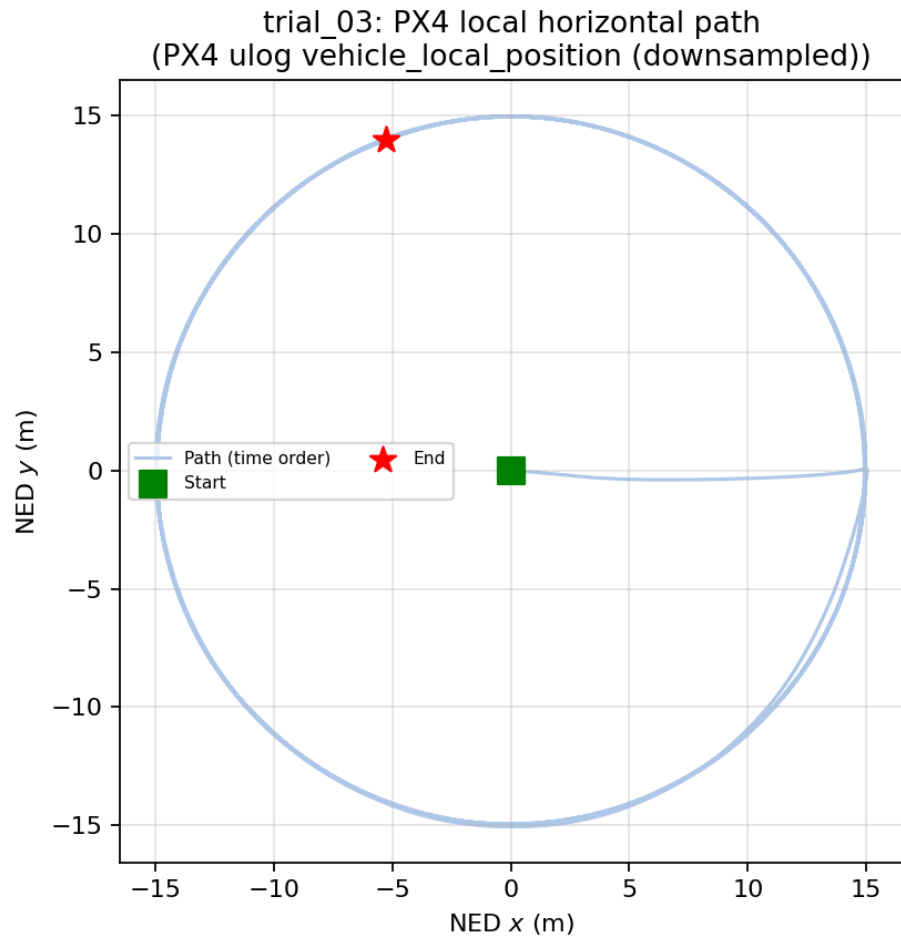


Figure 7: Trial 3: horizontal NED path from PX4 ulog.

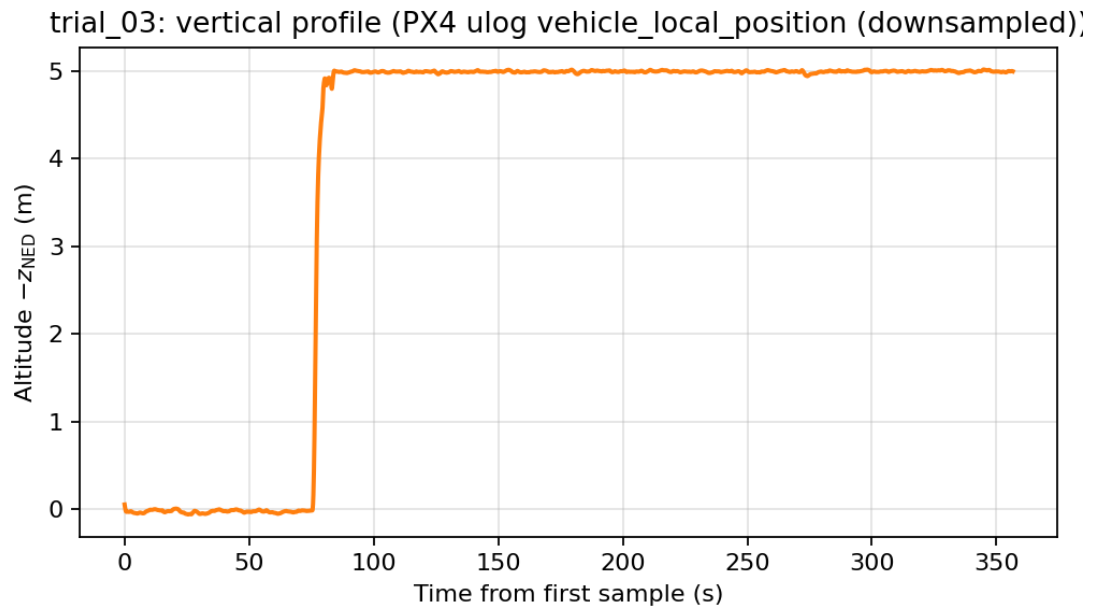


Figure 8: Trial 3: altitude profile from PX4 ulog.

8 Discussion

8.1 What worked or improved materially

- Three-terminal bring-up with consistent DDS domain and documented Micro XRCE binary resolution.
- Offboard arming and takeoff after intrinsics, local position, and setpoint streaming gates.
- RGB-D synchronization feeding the mission state machine.
- RTAB-Map ingestion after QoS and frame alignment; growing `rtabmap.db` across trials.
- Automated trial folders with ULGs (local), databases, ROS snapshots, and ulog-derived trajectory figures matching the tuned circular survey.
- Mission log capture improved with `PYTHONUNBUFFERED=1` and flush on completion.

8.2 Scope beyond intermediate mission

- **Extra credit — rover:** not evaluated in this PDF (cylinder-focused implementation).
- **Extra credit — dense mesh export:** optional per course instructions; RTAB-Map databases in each trial support offline export.

8.3 Engineering notes

- Micro XRCE or bridge restarts may be needed after aggressive kills; README documents cleanup.
- Earlier vision tuning iterations showed false-positive SERVO streaks; current depth gates and clamps address this (Section 2.3).

8.4 Issues encountered and resolutions

- **DDS discovery / no PX4 publishers:** enforce `ROS_DOMAIN_ID=0`; verify with `ros2 topic info /fmu/out/vehicle_local_position --qos-reliability best_effort`.
- **px4_msgs / uXRCE entities:** pin messages to course PX4 and rebuild.
- **RTAB-Map no images:** reliable QoS on images and `camera_info`.
- **Stale Python nodes:** `colcon build`; launches use `setup.py` console scripts.
- **Micro XRCE libraries:** `LD_LIBRARY_PATH` exports in `cylinder_landing.launch.py`.
- **Agent port:** prefer 8889; avoid mid-flight fuser kill on the agent port.
- **Sparse logs in collectors:** unbuffered Python and flush at mission end.

8.5 Submission checklist

1. PDF report with rubric mapping; trajectory metrics from PX4 ulog as in Section 7; cite mission text where stdout captures phases or footers.
2. Repository with reproducible instructions (`README.md`, scripts).

3. Three labeled trial folders under `run_artifacts/` (transcripts + DB committed; copy `.ulg` locally per `MANIFEST.md`).
4. Optional: demonstration video, mesh + Git LFS, rover extensions per course EC.

9 Conclusion

We delivered an integrated PX4-ROS 2-Gazebo pipeline with RGB-D perception, RTAB-Map SLAM wiring, ArUco-assisted landing logic, cylinder catalog gating, SERVO safety limits, and scripted trial collection. DDS, QoS, transform, and vision integration support repeatable SITL operation; three labeled trials archive transcripts, RTAB-Map databases, and (locally) PX4 ULGs (Table 2), with Section 7 confirming the circular survey and altitude hold from `vehicle_local_position`. Extra-credit rover and mesh objectives remain distinct extensions beyond the intermediate cylinder deliverable described here.

A Build and run commands

```
# One-time / after changes to entry points or Python modules
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --packages-up-to terrain_mapping_drone_control --symlink-install
source install/setup.bash

# Terminal 1
source ~/ros2_ws/src/terrain_mapping_drone_control/scripts/source_assignment3_env.sh
bash ~/ros2_ws/src/terrain_mapping_drone_control/scripts/terminal1_sim.sh

# Terminal 2 (after sim is up)
source ~/ros2_ws/src/terrain_mapping_drone_control/scripts/source_assignment3_env.sh
bash ~/ros2_ws/src/terrain_mapping_drone_control/scripts/terminal2_rtabmap.sh

# Terminal 3
source ~/ros2_ws/src/terrain_mapping_drone_control/scripts/source_assignment3_env.sh
bash ~/ros2_ws/src/terrain_mapping_drone_control/scripts/terminal3_mission.sh
```

B Automated trial batch

```
cd ~/ros2_ws/src/terrain_mapping_drone_control
# Optional: HW3_TRIALS=3 HW3_MAX_TRIAL_SEC=2400 python3 scripts/collect_clean_trials.py
python3 scripts/collect_clean_trials.py
```

C Figure regeneration (matplotlib + pyulog)

```
pip install matplotlib pyulog # if needed
cd ../terrain_mapping_drone_control
python3 scripts/plot_mission_report_figures.py
# Uses TELEM lines when present; otherwise trial_XX/latest_px4.ulg (local).

cd ../terrain_mapping_drone_control/report
pdflatex HW3_final_report.tex
pdflatex HW3_final_report.tex # second pass for references
```

References

1. PX4 Autopilot and px4_msgs: <https://px4.io/>
2. ROS 2 documentation: <https://docs.ros.org/>
3. RTAB-Map: <http://introlab.github.io/rtabmap/>